VISHWAKARMA INSTITUTE OF TECHNOLOGY, PUNE

**ASSIGNMENT NO: 02(a)**

---

**NAME: Ganesh Rayphale**

**PRN NO: 12411769**

**ROLL NO: 15**

**COURSE: COMPUTER SCIENCE AND ENGINERRING (IOTCSBT)**

**SUBJECT: AI**

---

Title :- Breadth-First Search (BFS) Algorithm for the 8-Puzzle Problem.

## Theory :-

**Breadth-First Search (BFS)**

**What is Breadth-First Search (BFS)?**

**Breadth-First Search (BFS)** is a graph traversal and search algorithm that explores all neighboring nodes at the current level before moving to the next level. It follows the **FIFO (First In, First Out)** principle by using a **Queue** data structure.

BFS is commonly used to find the **shortest path** in an unweighted graph or to search through state-space problems such as the **8-Puzzle**, maze solving, and network routing.

---

**Definition**

Breadth-First Search (BFS) is an uninformed search algorithm that starts from a source node and explores all its neighboring nodes first. It then visits the neighbors of those nodes, continuing level by level until the goal node is found or all reachable nodes have been explored.

---

**Working of BFS**

1. Start from the initial node (or initial puzzle state).
2. Mark the initial node as visited.

3. Insert the initial node into a queue.
4. Remove the front node from the queue.
5. Check whether it is the goal state.
6. If it is the goal, stop the algorithm.
7. Otherwise, generate all valid neighboring nodes.
8. Add every unvisited neighbor to the queue and mark it as visited.
9. Repeat the process until the queue becomes empty or the goal is found.

---

**Algorithm**

Step 1: Create an empty queue.

Step 2: Insert the initial node into the queue.

Step 3: Mark the initial node as visited.

Step 4: While the queue is not empty:

a) Remove the front node.

b) Check if it is the goal.

c) If yes, stop.

d) Otherwise, generate all neighboring nodes.

e) Add every unvisited neighbor to the queue.

Step 5: Repeat until the goal is found.

---

**Data Structure Used**

- **Queue (FIFO):** Stores the nodes to be explored.
- **Visited Set:** Keeps track of already visited states to avoid revisiting them.

---

**Advantages of BFS**

- Finds the shortest path in an unweighted graph.
- Simple and easy to implement.
- Guarantees finding a solution if one exists.
- Suitable for problems where the minimum number of moves is required.

---

**Disadvantages of BFS**

- Requires a large amount of memory.

- Becomes slow for very large search spaces.
- Stores all generated nodes in memory.

---

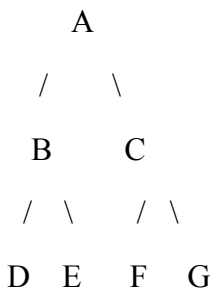
**Applications of BFS**

- 8-Puzzle Problem
- Maze Solving
- Robot Navigation
- GPS Route Finding
- Social Network Friend Search
- Web Crawlers
- Computer Networks

---

**Example of BFS**

Consider the following graph:




Suppose we start BFS from node **A**.

**Step 1**

Queue:

A

Visited:

A

---

**Step 2**

Remove **A**

Visit its neighbors **B** and **C**

Queue:

B C

Visited:

A B C

---

**Step 3**

Remove **B**

Visit **D** and **E**

Queue:

C D E

Visited:

A B C D E

---

**Step 4**

Remove **C**

Visit **F** and **G**

Queue:

D E F G

Visited:

A B C D E F G

---

**Final BFS Traversal**

A → B → C → D → E → F → G

The nodes are visited **level by level**, which is the main characteristic of Breadth-First Search.

---

**Time Complexity**

- **Time Complexity: O(V + E)**
    - **V** = Number of vertices (nodes)
    - **E** = Number of edges

---

**Space Complexity**

- **Space Complexity: O(V)**

because the queue and visited set may store all vertices in the worst case.

Program Code :-

```
import java.util.*;

class Node {
    String state;
    int moves;

    Node(String state, int moves) {
        this.state = state;
        this.moves = moves;
    }
}

public class EightPuzzleBFS {

    static String goal = "123456780";

    static int[] row = {-1, 1, 0, 0};
    static int[] col = {0, 0, -1, 1};

    static int bfs(String start) {

        Queue<Node> queue = new LinkedList<>();
        HashSet<String> visited = new HashSet<>();

        queue.add(new Node(start, 0));
        visited.add(start);

        while (!queue.isEmpty()) {

            Node current = queue.poll();

            if (current.state.equals(goal))
                return current.moves;

            int pos = current.state.indexOf('0');

            int x = pos / 3;
            int y = pos % 3;

            for (int i = 0; i < 4; i++) {

                int nx = x + row[i];
                int ny = y + col[i];

                if (nx >= 0 && nx < 3 && ny >= 0 && ny < 3) {

                    char[] arr = current.state.toCharArray();
```

```
                    char temp = arr[x * 3 + y];
                    arr[x * 3 + y] = arr[nx * 3 + ny];
                    arr[nx * 3 + ny] = temp;

                    String next = new String(arr);

                    if (!visited.contains(next)) {

                        visited.add(next);
                        queue.add(new Node(next, current.moves + 1));
                    }
                }
            }
        }

        return -1;
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.println("Enter the puzzle (9 digits, use 0 for blank):");

        String start = "";

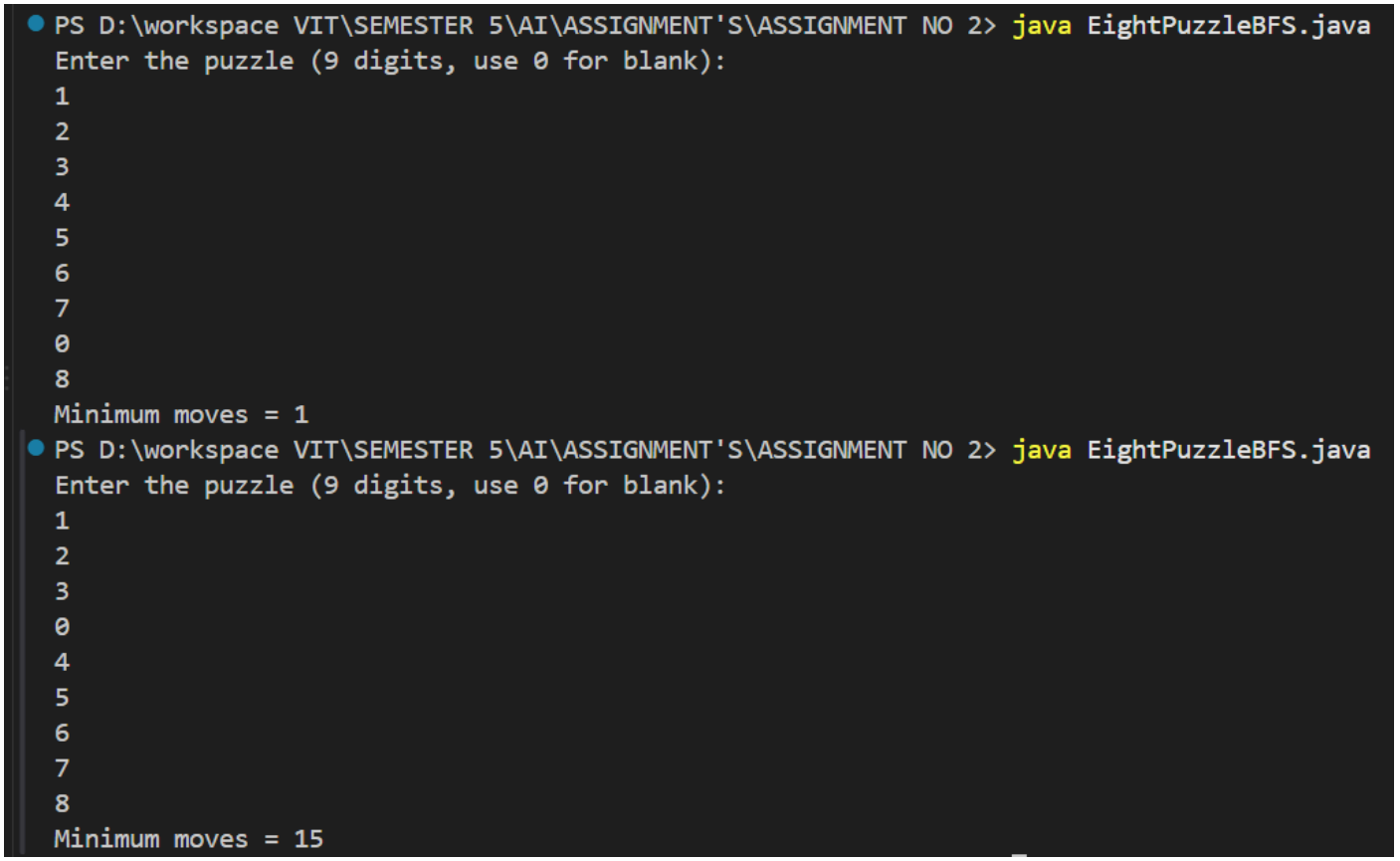
        for (int i = 0; i < 9; i++) {
            start += sc.next();
        }

        int result = bfs(start);

        if (result == -1)
            System.out.println("Puzzle cannot be solved.");
        else
            System.out.println("Minimum moves = " + result);

        sc.close();
    }
}
```

Output :-




Conclusion :-

Breadth-First Search (BFS) is a fundamental graph traversal algorithm that explores nodes level by level using a queue. It is widely used for finding the shortest path in unweighted graphs and solving state-space problems such as the 8-puzzle, maze solving, and network routing. Although BFS requires more memory than some other search algorithms, it guarantees the shortest solution when one exists, making it highly effective for many practical applications.